

PREPARACAO PARA A DEFESA

100 perguntas & respostas

Computador de Bordo Determinístico para CubeSats — Projeto e
Seminário (ISEL)

Como usar este guia

Este documento reúne 100 perguntas prováveis e respostas curtas de defesa, alinhadas com o relatório e com a implementação real do sistema. Está organizado em oito temas: **(1)** desafios de engenharia e decisões de design (estilo "qual o maior desafio"), **(2)** fundamentos e estado da arte, **(3)** requisitos e arquitetura, **(4)** implementação de HAL e drivers, **(5)** aplicação, periféricos e telemetria, **(6)** OTA e bootloader, **(7)** segmento de solo e **(8)** testes e validação. As respostas são propositalmente concisas para memorizar o essencial; em cada uma sabes ir mais fundo se o professor puxar.

Dica de defesa. Para perguntas de "maior desafio" ou trade-off, responde sempre em três tempos: o problema, a decisão que tomaste, e o custo/benefício dessa decisão. Isso mostra critério de engenharia, que é o que esse tipo de pergunta procura.

1 · Desafios de engenharia e decisões de design

1 Qual foi o maior desafio de engenharia do projeto?

Garantir determinismo temporal sem RTOS. Foi preciso fatiar toda a lógica (drivers, missão, OTA) em máquinas de estados não-bloqueantes que avançam um passo por iteração do super-loop, de modo que nenhuma operação de I/O trave o ciclo e que o pior caso de execução (WCET) seja limitado e mensurável. Isso obrigou a reescrever cada driver como FSM com timeout e callback e a validar o WCET com o contador de ciclos DWT.

2 Porquê bare-metal em vez de um RTOS?

Os requisitos não-funcionais exigem previsibilidade do WCET e proibição de alocação dinâmica. Um RTOS traz overhead de scheduler, risco de deadlock e inversão de prioridade, e maior footprint; num OBC com recursos limitados e sujeito a SEUs, o super-loop cooperativo dá controlo total do tempo de execução e um consumo de RAM fixo determinado em compilação.

3 Qual a principal desvantagem do bare-metal que tiveste de gerir?

A ausência de preempção: uma tarefa que demore atrasa todas as outras. Mitiguei-a garantindo que nenhum estado bloqueia (tudo é "faz um passo e volta"), impondo timeouts em todo o I/O e medindo o WCET de cada função do super-loop para confirmar a margem face ao deadline de 1 ms.

4 Porquê máquinas de estados finitas em vez de programação sequencial?

Porque permitem execução não-bloqueante e determinística: cada FSM guarda o seu estado entre iterações e progride um passo de cada vez, evitando esperas ativas. Além disso, as FSM são formalmente analisáveis e mapeiam-se diretamente para os requisitos (modos de missão, protocolos), o que facilita o teste e a rastreabilidade.

5 Porque proíbes malloc e heap?

Para eliminar fragmentação, falhas de alocação em runtime e, sobretudo, o risco de corrupção de apontadores de heap por SEU, que seria catastrófica. Com alocação estática, o consumo de RAM é conhecido em compilação e constante durante toda a missão.

6 Porquê TMR em software e não hardware rad-hard?

Hardware rad-hard é caríssimo e está fora do âmbito de um projeto académico com uma placa comercial. O TMR em software dá tolerância a SEU nos dados críticos a custo zero de hardware, à troca de triplicar a RAM e gastar algum CPU por ciclo, um compromisso adequado ao contexto.

7 Porquê CRC32 e não uma assinatura criptográfica no OTA?

O CRC32 garante integridade (deteta corrupção do firmware), que era o requisito. Assinaturas assimétricas (ECDSA/Ed25519) dariam autenticidade contra adversários ativos, mas consomem ciclos, aumentam o footprint na flash e prolongam o arranque num Cortex-M. O relatório identifica isto como trabalho futuro, não como requisito atual.

8 Qual o compromisso na escolha de 9600 baud na UART?

Baixo débito troca velocidade por robustez: tolera melhor ruído e atenuação, coerente com um enlace de rádio real. O custo é um OTA moroso (com ACK por pacote), aceitável porque a atualização é uma operação rara e não crítica em tempo.

9 Porque separaste o SPI do I2C com o spi_turn?

Porque a unidade de arbitragem correta é o barramento, não o periférico. Os 5 sensores I2C partilham um bus e já são serializados pela flag bus_locked; o propulsor SPI é um bus independente. Alternar entre o grupo I2C e o SPI evita deixar transações I2C a meio (que dessincronizavam o escravo e travavam o bus) sem perder a coerência temporal do snapshot de telemetria.

10 Qual foi o bug mais difícil de resolver?

O lockup de I2C: o barramento morria e só voltava com power cycle. A causa era uma transação I2C interrompida a meio, com o escravo a segurar o SDA. Resolvi-o com o spi_turn (prevenção) e com uma rotina de bus recovery (9 pulsos de clock manuais mais STOP) que liberta o escravo em software, substituindo o "desligar a placa".

11 Como garantes que uma atualização OTA não entijola o satélite?

Com três camadas de defesa: o bootloader vive isolado nos primeiros 64 KB e nunca é sobrescrito; verifica o CRC32 da imagem na flash externa antes de tocar na app interna; e só escreve a flag (magic) por último. Falha antes disso arranca a versão anterior; falha durante a cópia deixa a flag pendente e retenta no próximo reset.

12 Qual a maior limitação atual do sistema?

O débito das ligações série (9600 baud e I2C a 25 kHz) e a ausência de autenticação criptográfica no OTA. Identifiquei também que o ramo de falha da cópia no bootloader depende de um reset externo/watchdog para retentar, algo a robustecer.

13 Se recomeçasses, o que farias diferente?

Consideraria DMA para as transferências série (libertando o CPU e subindo o débito sem perder determinismo), unificaria os defines duplicados (por exemplo QSPI_RETRY_MAX) e adicionaria um contador de tentativas e um watchdog ativo no bootloader para tornar o retry da cópia autónomo.

14 Qual foi a decisão de arquitetura de que mais te orgulhas?

A separação rigorosa em camadas com FSM não-bloqueantes e API uniforme por barramento. Deu-me código testável com mocks, portátil (só a HAL depende do MCU) e robusto (timeouts, bus recovery, TMR), e é o que sustenta o determinismo do sistema.

2 · Fundamentos e estado da arte

15 O que é firmware e o que o distingue de software de aplicação?

É software residente em memória não-volátil, executado diretamente pelo microprocessador para controlar o hardware. Distingue-se por três traços: execução sem SO hospedeiro (acesso direto a registos), determinismo temporal (resposta limitada e previsível) e gestão estática de recursos (RAM fixa em compilação).

16 O que é um autómato finito e porque é central no teu firmware?

É um modelo de computação com um número finito de estados e transições disparadas por condições. No firmware, cada driver e a lógica de missão são autómatos: guardam estado entre iterações e avançam um passo por ciclo, o que dá execução não-bloqueante, determinística e formalmente analisável.

17 Que níveis de autómatos existem no sistema?

Uma FSM de topo (ciclo de vida: Boot, Stabilize, Recovery, Mission Lifecycle), a FSM de modos operacionais (6 modos) dentro do Mission Lifecycle, e FSM de baixo nível em cada driver (I2C, SPI, USART TX/RX, QSPI) e na receção OTA. É uma hierarquia de autómatos.

18 Bare-metal vs RTOS, quando escolherias cada um?

Bare-metal quando o determinismo e o footprint mínimo são críticos e o número de tarefas concorrentes é gerível por um super-loop, como neste OBC. Um RTOS compensa quando há muitas tarefas com prioridades e prazos distintos que justificam preempção, aceitando o overhead e o risco acrescido.

19 Que frameworks de software de voo existem e porque não os usaste?

Existem frameworks institucionais (por exemplo NASA cFS) e open-source (por exemplo KubOS). Não os usei porque trazem uma camada de abstração e dependências pesadas para o âmbito do projeto; o objetivo era dominar e demonstrar a construção bare-metal determinística de raiz, com controlo total sobre WCET e memória.

20 O que é um SEU e porque é relevante no espaço?

Um Single Event Upset é a inversão de um bit provocada por radiação cósmica. Em órbita é frequente e pode corromper dados ou apontadores; por isso uso TMR nos dados e proíbo heap, cuja corrupção seria catastrófica.

21 O que é o WCET e porque importa aqui?

Worst-Case Execution Time, o tempo máximo que uma função pode demorar. Num OBC real-time, exceder o deadline pode originar dados de atitude desatualizados ou perda de janela de comunicação; por isso medi o WCET de cada função do super-loop e confirmei margem face ao deadline de 1 ms.

22 O que torna um sistema determinístico?

O tempo de resposta a um evento é previsível e limitado. Consigo-o com super-loop cooperativo (ordem fixa de execução), FSM não-bloqueantes (sem esperas de duração indeterminada), timeouts em todo o I/O e ausência de heap e de scheduler.

23 Porque é a gestão estática de memória particularmente relevante no espaço?

Porque sem heap o consumo de RAM é fixo e conhecido, eliminando falhas de alocação, e não há apontadores de heap cuja corrupção por SEU levaria a comportamento indefinido. A memória estática é mais fácil de proteger (por exemplo com TMR) e de raciocinar.

24 Como posicionas este projeto face ao estado da arte?

Como uma implementação bare-metal determinística, de raiz, que cobre a cadeia completa (OBC, bootloader OTA, segmento de solo e bancada HIL) com mecanismos de robustez (TMR, CRC, rollback) normalmente vistos em frameworks maiores, mas aqui construídos e compreendidos ao nível do registo.

3 · Requisitos e arquitetura do sistema

25 Como organizaste a arquitetura do firmware?

Em quatro camadas hierárquicas: HAL (acesso a registos do MCU), Drivers (FSM não-bloqueantes por barramento), Periféricos (módulos por dispositivo que fazem parsing) e Aplicação (missão, modos, telemetria). Cada camada só conhece a imediatamente abaixo.

26 Que vantagem prática te deu a separação em camadas?

Permite trocar de MCU alterando só a HAL, testar a lógica com mocks (`USE_REAL_HW=0`) e isolar falhas: por exemplo `eps.c` só conhece o `i2c_driver`, e o driver só conhece o `hal_i2c`. Reduz acoplamento e facilita a rastreabilidade requisito-código.

27 Descreve o super-loop cooperativo.

Após o boot, o `main()` entra num `while(1)` que chama `mission_lifecycle()` indefinidamente; este faz avançar as FSM de todos os subsistemas um passo por iteração, sem bloquear. É cooperativo porque cada tarefa devolve o controlo voluntariamente e rapidamente.

28 Quais são os seis modos operacionais?

Nominal, comunicação, atualização remota (OTA), seguro (safe), consumo ultra-baixo e fim de vida (decommissioning). As transições são automáticas, decididas por `getMode()` com base na saúde do sistema (tensão, corrente, temperatura, bateria).

29 Como decides entrar em safe mode?

A função `isSystemSafe()` compara tensão (3,1 a 6,5 V), corrente (1 a 2 A), temperatura (-10 a 60 graus) e bateria ($\geq 30\%$) com os limiares de `mission.h`; se algum sair da janela, `getMode()` força `safe_mode` a partir de qualquer modo operacional.

30 O que diferencia safe mode de ultra-low-power?

Safe mode reduz atividade mas mantém telemetria mínima e ativa-se por qualquer parâmetro de saúde fora dos limites. Ultra-low-power entra quando a bateria fica crítica ($<15\%$) e corta consumos ao mínimo; dele só se sai recuperando bateria ou, por timeout de missão, para fim de vida.

31 Porque exiges bateria acima de 70% para entrar em OTA?

Porque uma atualização interrompida por falta de energia a meio da cópia para a flash interna é o cenário mais perigoso. Exigir mais de 70% garante margem para completar a transferência e a gravação sem risco de corte de energia no pior momento.

32 Como é o ciclo de vida de topo (Boot, Stabilize, Recovery)?

No arranque o sistema faz Boot (inicialização e auto-testes), Stabilize (estabiliza clocks e periféricos) e só depois entra no Mission Lifecycle; há um caminho de Recovery para falhas de inicialização, com um limite de tentativas (`MAX_INIT_RETRIES=5`) antes de reset.

33 Como partilham os módulos os dados dos sensores?

Através de estruturas globais (eps, imu, gnss, pressure, temperature, propulsor, ttc), que são a fonte de verdade. Cada periférico escreve na sua struct no callback; a aplicação lê-as para decidir modos e compor telemetria. Estas structs estão sob proteção TMR.

34 Que riscos identificaste e como os mitigaste?

SEU com TMR nos dados; lockup de barramento com bus recovery e spi_turn; I/O que bloqueia com FSM não-bloqueantes e timeout; OTA falhado com bootloader com CRC e rollback/retry; falha de inicialização com limite de retries e reset. Cada risco tem uma mitigação implementada e testada.

35 Quais são os requisitos não-funcionais mais determinantes?

Previsibilidade do WCET, proibição de alocação dinâmica, e robustez a SEU e a falhas de energia. São eles que justificam bare-metal, FSM, memória estática e o bootloader com rollback.

36 Que interfaces de comunicação suporta o OBC?

I2C (sensores GNSS, IMU, EPS, pressão, temperatura), SPI (propulsor), USART/UART (TT&C com a estação e UART de debug) e QSPI (flash externa). Cada uma tem um driver não-bloqueante dedicado.

4 · Implementação: HAL e drivers

37 O que faz a camada HAL?

Isola o silício: configura clocks/PLL, GPIO, SysTick e os periféricos TWIHS (I2C), SPI, USART e QSPI por acesso direto a registos. Expõe primitivas (start, send_byte, get_ack, is_busy) que os drivers usam sem conhecerem endereços de registo.

38 Descreve a FSM do driver I2C.

Estados: STARTING, SELECT_MODE, RESTART/WAIT_RESTART, READ/WRITE, WAIT_RX/WAIT_TX e STOP. Suporta leitura por registo com repeated start, verifica ACK em cada passo, aplica timeout (I2C_TIMEOUT_MAX) e chama callback(0/-1) no fim.

39 Como funciona uma leitura por registo em I2C?

START; envia endereço com bit de escrita; ACK; envia o registo; ACK; repeated START; envia endereço com bit de leitura; ACK; lê N bytes dando ACK entre bytes e NACK no último; STOP. É o caminho use_reg=1 usado pelo EPS, IMU e outros.

40 Como é feito o bus recovery no I2C?

Após 3 falhas seguidas: SWRST ao TWIHS, tomo os pinos como GPIO, gero até 9 pulsos de clock em SCL (verificando se o SDA sobe para parar cedo), gero um STOP manual e devolvo os pinos ao periférico reiniciando-o. Liberta um escravo preso sem power cycle.

41 Porque exatamente 9 pulsos de clock no recovery?

Porque um byte I2C tem 8 bits mais o ACK; 9 pulsos garantem que o escravo termina qualquer transação pendente e larga a linha SDA, permitindo depois um STOP válido para repor todos os escravos em idle.

42 Descreve a FSM do driver SPI.

IDLE, CS_LOW, TRANSFER, WAIT_TX, WAIT_RX, CS_HIGH e STOP. Controla explicitamente o chip-select, transfere em full-duplex (envia dummies para puxar dados), aplica timeout e devolve o resultado por callback.

43 Porque o SPI é full-duplex e o que envias como dummy?

No SPI cada clock desloca um bit em cada sentido em simultâneo; para receber tenho de transmitir. Quando só quero ler, envio bytes dummy (0x00) para gerar clock e receber os dados do propulsor.

44 Como estão organizados os drivers USART?

Em duas FSM independentes, TX e RX. A TX envia byte a byte quando o registo fica livre; a RX recebe byte a byte por interrupção (RXRDY) com buffer circular, sem bloquear. Isto permite receber comandos enquanto se transmite telemetria.

45 Como resolves o desalinhamento de bytes na UART no arranque?

Com um protocolo de sincronização: o OBC procura a sequência [0xAA][0x55][0xAA][0x55]; ao detetá-la responde [0x55][0xAA][0x55][0xAA] e passa a modo normal. Se após 32 bytes não houver sync, assume que o Pico já está em modo normal e entra direto em receção, evitando deadlock.

46 Descreve a FSM do driver QSPI.

IDLE, WRITE_ENABLE, CHECK_WEL, SEND_COMMAND, WRITING/READING, WAIT_BUSY e IDLE, com ramo de erro. A leitura salta o Write-Enable; escrita e erase exigem WREN e confirmação do bit WEL antes do comando.

47 Porque verificas o bit WEL antes de escrever no QSPI?

Porque a flash só aceita escrita/erase com o Write-Enable-Latch ativo; se o WREN não pegou, a operação seria ignorada silenciosamente. Leio o status register e, se o WEL não estiver ativo, retento o WREN até um máximo antes de declarar erro.

48 O QSPI faz bus recovery como o I2C?

Não. O QSPI faz retry do WREN, timeout em cada estado (QSPI_TIMEOUT_MAX) e abort limpo (callback de erro, volta a IDLE). Como é master único, a falha típica não é uma linha presa mas uma operação que não confirma, daí retry e timeout em vez de colocar linhas.

49 Porque todos os drivers usam callbacks?

Porque a conclusão de uma operação assíncrona acontece muitos ticks depois de ser iniciada; o callback notifica a camada de periférico (0=sucesso, -1=erro) para fazer o parsing ou tratar o erro, mantendo o fluxo não-bloqueante.

50 Como evitas que um sensor avariado bloqueie o sistema?

Cada operação de I/O tem um contador de timeout; se o hardware não responde dentro do limite, a FSM aborta a transação, marca erro e devolve o controlo. O super-loop continua e o sensor falhado apenas não atualiza o seu valor nesse ciclo.

51 Como calculas o clock do I2C a partir de MCK?

A partir de MCK=150 MHz e da velocidade desejada (25 kHz), calculo os divisores CLDIV/CHDIV/CKDIV do registo CWGR do TWIHS pela fórmula do datasheet ($T_{low} = (CLDIV \times 2^{CKDIV} + 3) / MCK$), escolhendo o CKDIV para o CLDIV de 8 bits chegar.

52 Porque o I2C corre a 25 kHz e não a 400 kHz?

Por robustez na bancada e margem de temporização; 25 kHz reduz erros de bit e simplifica timeouts. O custo é menos débito, aceitável porque os sensores são poucos e amostrados a baixa cadência; subir para 400 kHz é uma otimização futura.

53 Como serializa os 5 sensores no mesmo barramento I2C?

Com a flag global `bus_locked` no driver: só uma transação I2C ocupa o bus de cada vez, as restantes esperam. Assim posso disparar as 5 leituras juntas que elas executam em sequência limpa, sem colidirem.

54 Onde está a lógica específica de cada sensor?

Na camada de periféricos: cada módulo (`eps.c`, `imu.c`) configura o handle do driver, inicia a leitura assíncrona e, no callback, converte os bytes crus em grandezas físicas (parsing e escala), escrevendo na struct global. Não toca no hardware diretamente.

5 · Implementação: aplicação, periféricos e telemetria

55 O que faz `sensors_tick()` em cada iteração?

Regista as structs sob TMR (na 1ª vez), faz `seu_data_scrub()` (votação e reparação), avança as FSM de todos os periféricos e da memória externa, e faz `seu_data_commit()` (consolida escritas legítimas nas 3 cópias). É o coração do ciclo de aquisição.

56 Porque o scrub e o commit do TMR envolvem as leituras?

Para que o scrub repare corrupções por SEU antes de usar os dados, e o commit consolide apenas as escritas legítimas feitas pelos callbacks nesse tick nas três cópias, sem que uma alteração real seja revertida pela votação.

57 Como garantes que as 3 cópias TMR não são atingidas pelo mesmo evento?

Vivem em três bancos estáticos separados que o linker coloca em regiões distintas da SRAM. A separação física reduz a probabilidade de um único evento atingir mais do que uma cópia; é uma mitigação por layout, não uma garantia de hardware.

58 Como converte o EPS os bytes em tensão e corrente?

Lê 2 bytes do registo 0x09: `voltage = buf[0]/10` e `current = buf[1]/100`. São escalas fixas escolhidas para os intervalos esperados (por exemplo 0xAA dá 17,0 V).

59 Porque o buffer do GNSS e do IMU é 18 bytes e o do EPS 2?

Porque o tamanho depende do número de grandezas e da codificação: o EPS tem 2 valores de 1 byte; o GNSS envia 4 floats (16 B usados) e o IMU 9 eixos. Os 18 bytes de GNSS/IMU são um buffer generoso (o parsing usa 16 e 9 respetivamente).

60 Descreve a estrutura `ttc_data_t`.

Tem 84 bytes: `eps` (8), `temp` (4), `press` (4), `gnss` (16), `imu` (36, 9-DOF), `doppler` (4), `ranging` (2), `current_state` (1), `ota_active` (1), `last_command` (4) e `cmd_status` (4). Agrega toda a telemetria e o estado num só bloco.

61 Qual o formato da trama de telemetria enviada?

75 bytes: marcador 0x20 (`CMD_REQUEST_DATA`), 18 floats big-endian (tensão, corrente, lat, lon, alt, velocidade, 3 de aceleração, 3 de giroscópio, 3 de magnetómetro, pressão, temperatura, doppler), o estado do TT&C e um checksum XOR de todos os bytes anteriores.

62 Porque big-endian e checksum XOR na telemetria?

Big-endian por convenção de rede e leitura consistente no lado de solo. O XOR é baratíssimo em CPU e suficiente para detetar erros de bit isolados numa telemetria de alta cadência; onde a integridade é crítica (OTA) uso CRC32.

63 Como e onde guardas telemetria a bordo?

Na flash externa, na região de logs, via ExtMem_SaveTelemetryAsync, que grava a struct como bloco respeitando o limite de 256 bytes por página; só inicia se a memória estiver IDLE, senão salta o ciclo.

64 Como o propulsor comunica e valida os dados?

Por SPI: o OBC envia um comando (0x01 pedir telemetria, 0x02 abrir válvula) e lê uma trama de 9 bytes (status, pressão, temperatura, impulso, válvula, checksum XOR). O parsing só aceita a trama se o XOR bater certo.

65 Como alternas leituras SPI e I2C sem as sobrepor?

Com a flag spi_turn em sensors_read_all(): num ciclo disparo as 5 leituras I2C, no seguinte a leitura SPI do propulsor. Isto dá a cada barramento um ciclo limpo, mantém a coerência do snapshot I2C e evita transações a meio.

66 Que comandos de solo o OBC aceita?

CMD_NONE (0x00), CMD_START_OTA (0x10), CMD_END_OTA (0x11) e CMD_REQUEST_DATA (0x20), recebidos em tramas de 4 bytes pela UART de TT&C.

6 · Subsistema OTA e bootloader

67 Como está organizada a memória para o OTA?

Flash interna: bootloader em 0x00400000 a 0x0040FFFF (64 KB) e aplicação a partir de 0x00410000 (cerca de 1984 KB). Flash externa S25FL116K (2 MB): configuração (64 KB), logs (960 KB), metadados OTA (0x100000, 4 KB) e imagem OTA (0x101000, 1020 KB).

68 Porque separaste os metadados do firmware em setores distintos?

Porque o erase da flash é por setor de 4 KB. Se metadados e firmware partilhassem setor, apagar a flag OTA destruiria os primeiros 3840 bytes do firmware. Setores separados permitem limpar a flag sem tocar na imagem.

69 O que contém a estrutura de metadados OTA?

256 bytes (packed): magic (flag de validade), firmware_size, firmware_crc32, fw_version e padding. Os 256 bytes coincidem com uma página da S25FL116K, tornando a escrita dos metadados atômica ao nível do hardware.

70 Porque o magic funciona como flag atômica?

Porque cabe numa página e é escrito de uma vez: ou fica 0xAB12CD34 (imagem pendente válida) ou 0xFFFFFFFF (sem OTA). Não há estado intermédio parcialmente escrito que o bootloader possa interpretar mal.

71 Descreve o fluxo do bootloader.

Após reset: inicializa QSPI; lê metadados; verifica magic; valida firmware_size; verifica CRC32 da imagem externa; copia para a flash interna; apaga a flag; salta para a aplicação. Cada passo tem um caminho de falha seguro.

72 Se o OTA falhar durante a receção, o satélite recupera a versão antiga?

Sim, sempre. Nessa fase a app interna nunca é tocada, os pacotes vão para a flash externa e a flag só é escrita no fim, após CRC. Qualquer falha antes disso deixa a flag por escrever e o bootloader arranca a app antiga intacta.

73 E se falhar durante a cópia para a flash interna?

Aí a app antiga já foi apagada e não é recuperável. Mas a imagem nova continua íntegra na flash externa e a flag fica pendente, pelo que o bootloader repete a cópia no próximo reset. É retry para a frente, não rollback para a antiga.

74 O que garante que a cópia não é aplicada com firmware corrompido?

O CRC32 é verificado duas vezes: na flash externa antes de apagar a app interna, e novamente na flash interna após a cópia. Só com ambos corretos é que a flag é limpa e o salto acontece.

75 Porque não apagas os metadados em falha de escrita, mas apagas em falha de CRC?

Falha de escrita é potencialmente transitória: manter a flag permite retentar no próximo reset. Falha de CRC significa imagem má; apagar a flag evita um ciclo infinito a tentar aplicar firmware corrompido.

76 Como o bootloader salta para a aplicação?

Faz o handoff padrão Cortex-M: desativa IRQs e SysTick, escreve o SCB VTOR com o endereço da app, barreiras dsb/isb, carrega o MSP do vetor[0] da app e faz BX para o Reset_Handler (vetor[1]), em assembly para não deixar restos de stack.

77 Como o bootloader sabe que existe uma aplicação válida?

Verifica o vetor de reset da app (não pode ser 0x00000000 nem 0xFFFFFFFF). Se inválido, entra num loop de segurança à espera de programação por JTAG, em vez de saltar para lixo.

78 Como recebe a aplicação os pacotes OTA (otaMode)?

Uma sub-FSM grava os pacotes na flash externa: limpa a write-protection, apaga setores a pedido, escreve cada pacote no endereço FW+seq*128 e, no fim (pacote END), valida o CRC e escreve os metadados. Tem timeout para abortar sessões abandonadas.

79 Como funciona o protocolo ACK/NACK do OTA?

É stop-and-wait: por cada pacote válido o OBC envia ACK (0xAC); se a sync word falhar envia NACK (0x4E). O backend só envia o pacote seguinte após ACK, e retransmite (até 3 vezes, timeout de 3 s) em NACK ou timeout.

80 O que é o frame scanner do OTA e para que serve?

Depois do CMD_START_OTA, o OBC procura a sync word [0xAA][0x55] byte a byte antes de cada pacote de 134 bytes. Serve para realinhar o fluxo na transição de comandos de 4 bytes para pacotes de 134, já que os primeiros bytes podem ter sido consumidos.

81 O que impede o OBC de perder pacotes se a flash ainda está a escrever?

Back-pressure: se o buffer de staging ainda está ocupado, o OBC não envia ACK e descarta o pacote; o backend faz timeout e retransmite. O ritmo do envio fica limitado pela velocidade de escrita da flash, sem overflow.

82 Porque o bootloader é um projeto MPLAB independente?

Porque tem o seu próprio linker script, tabela de vetores e main(), e vive numa região de flash própria (0x00400000) que nunca é sobrescrita pelo OTA. Isto isola o agente de recuperação da aplicação que ele atualiza.

7 · Segmento de solo (dashboard)

83 Como está estruturado o dashboard de solo?

Em três camadas: backend Kotlin/Spring Boot (REST, WebSocket e série), frontend React/TypeScript e base de dados PostgreSQL. Recebe telemetria por porta série, persiste-a e difunde-a para o browser em tempo real.

84 Como o backend lê e decodifica a telemetria?

O ComPortService lê bytes crus da porta série e coloca-os numa fila; um segundo thread alimenta o codec escolhido (RAW binário, ASCII ou AUTO). Os codecs implementam a mesma interface (padrão Strategy) e uma factory escolhe-o por ligação.

85 Como está feita a segurança do dashboard?

Autenticação por JWT (JJWT) com um filtro (JwtAuthFilter) e controlo de acessos por papéis (RBAC: Role e Permission), além de auditoria das ações. Protege comandos sensíveis como o envio de OTA.

86 Como chega a telemetria ao browser em tempo real?

Por WebSocket: o SatelliteWebSocketHandler difunde mensagens tipadas (TELEMETRY, STATUS, LOG, OTA_PROGRESS). É push em vez de polling, o que dá atualização eficiente e imediata dos painéis.

87 Porque limitas a telemetria a 1 emissão por segundo?

A fonte pode debitar dezenas de tramas por segundo; um throttle de 1 por segundo evita inundar o WebSocket e a base de dados, mantendo sempre o valor mais fresco em memória e a vista fluida.

88 Como o dashboard envia firmware por OTA?

O OtaService suporta .bin (empacotado em pacotes de 134 bytes via a ponte) e .hex (Intel HEX linha a linha). Envia CMD_START_OTA, espera READY, envia os pacotes com ACK por pacote e retries, e termina com um pacote END com size, CRC e versão.

89 Como está pensada a extensibilidade do segmento de solo?

O transporte é abstraído: ComPortService e BluetoothService partilham interface, e para antenas bastaria um novo serviço com a mesma interface. Os dados chegam sempre pelo mesmo WebSocket, pelo que o frontend não precisa de mudar.

90 Que dados persistes em base de dados e porquê?

Telemetria, logs, estado dos subsistemas, utilizadores/papéis e auditoria, em PostgreSQL. Permite histórico, análise posterior e rastreabilidade das ações do operador, essencial para operação de missão.

8 · Testes e validação

91 Que estratégia de testes seguiste?

Uma suite com várias categorias: testes unitários (lógica dos drivers com mocks), validação em hardware (flash QSPI), medição de WCET e injeção de SEU para validar o TMR. Cada requisito está mapeado a uma validação numa matriz de rastreabilidade.

92 Como testaste o driver I2C sem hardware?

Com mocks (`USE_REAL_HW=0`): o HAL devolve dados simulados e permite exercitar a FSM (START, ACK, repeated start, leitura, timeouts, bus recovery) de forma determinística, validando as transições sem depender dos sensores reais.

93 Como validaste a flash externa QSPI?

Com testes de leitura e escrita na placa (3 de 3 casos aprovados, após corrigir o driver de escrita em Serial Memory Mode), confirmando erase, page program e read no S25FL116K real.

94 Como mediste o WCET?

Com o contador de ciclos DWT do Cortex-M7: instrumentei as funções do super-loop e confirmei que todas cumprem o deadline de 1 ms (12000 ciclos a 12 MHz) com cerca de 30% de margem de segurança.

95 Porque o deadline é 1 ms e o que significa a margem de 30%?

1 ms é a base de tempo escolhida para o ciclo de controlo; garantir que o pior caso do super-loop cabe nele assegura o determinismo. A margem de 30% dá folga para variações e futuras adições sem violar o deadline.

96 Como validaste a proteção TMR?

Por injeção de falhas: corrompo deliberadamente cópias das estruturas e observo o comportamento. Upsets simples são reparados por votação 2-de-3; um upset duplo é detetado, com degradação determinística em vez de comportamento indefinido.

97 O que é a matriz de rastreabilidade requisitos-validação?

Uma tabela que liga cada requisito (funcional e não-funcional) ao teste ou mecanismo que o valida, provando que cada requisito foi verificado e onde. É a evidência de cobertura do sistema.

98 O que é a validação HIL (hardware-in-the-loop) no teu projeto?

É testar o firmware real no MCU em ciclo fechado com hardware que simula o resto: dois Picos (um simula os sensores I2C e a estação via UART, outro simula o propulsor via SPI) e a flash QSPI real. Valida o sistema em condições próximas do real.

99 Porque usar dois Raspberry Pi Pico na bancada?

Para separar os domínios de barramento: um Pico responde aos 5 endereços I2C (via PIO) e faz de estação terrestre por UART; o outro simula o propulsor por SPI. Assim exercito I2C, UART e SPI em paralelo, como no sistema real.

100 Que trabalho futuro identificas?

Autenticação criptográfica do OTA (assinaturas Ed25519/ECDSA), uso de DMA para subir o débito série mantendo determinismo, watchdog e contador de tentativas para tornar o retry da cópia do bootloader autónomo, e subir as velocidades de I2C e UART após validação.